AI ACADEMY · LEVEL 2 · GRADE 7 · WEEK 33 · STUDENT LESSON

Build the Recommender

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Transparent Book Recommender

THE CONNECTED WEEK

Validate. Filter. Score. Apply the declared tie.

Build a reproducible baseline with complete local code and explicit exclusions.

Keep one habit: Compact selector → state trace → separate book ranking → complete pipeline → independent catalogue.

MAIN LESSON · SECTION 1

Can another learner reproduce this recommendation?

> Build the declared filter, score and tie policy, then compare expected and actual results.

Your learning goals

I can implement a **deterministic recommender**.

I can handle ties and no eligible options.

I can explain recommendations with their limits.

Recall a hard constraint

- Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

- An option violates a hard duration limit but has the best score. Should it win?

Your end goal

By the end, I can implement **deterministic** filtering, scoring and ties for a complete catalogue case.

▼ Pipeline extension · Name the stages

What are the steps between raw records and a recommendation?

> **My readiness explanation**

▼ Earlier wording, alternatives and picture transcripts

Your learning target

Remember before reading

Without opening the old lesson, explain one key idea from Project Problem and Success Test. Give an example and a mistake that the idea helps you avoid.

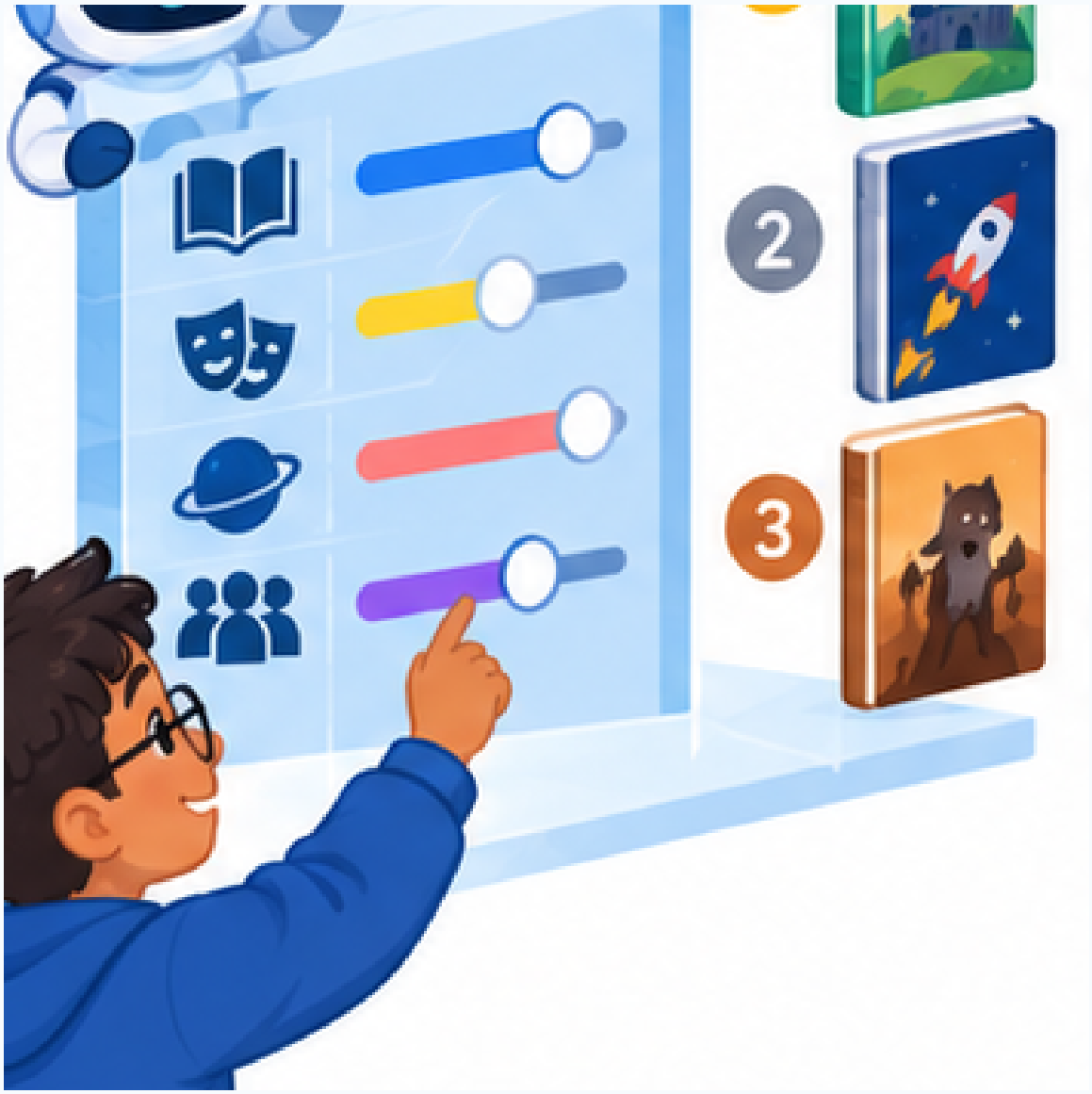
Without opening the old lesson, explain one key idea from Agents and Tools. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Predictive vs Generative. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your **explanation** that the **evidence** shows was wrong.

Week 33: Build the **Recommender**

A transparent **baseline** makes later improvements understandable.



THINKER MISSION

Implement a scoring table and produce ranked **output**s.

Thinker name and date:

> **Write or draw your reasoning and evidence.**

Readiness for the connected explanation

Readiness for Building the data pipeline

This week’s end goal

MAIN LESSON · SECTION 2

Implement a deterministic baseline

> Filtering, scoring and ties must follow the frozen contract.

The main activity-selector contract

Build a fictional activity **recommender** using last week's score. Require duration at most a supplied limit. On a tie choose the alphabetically first name.

This makes the outcome reproducible rather than dependent on **input** order. Assume unique names and validated numeric fields.

▼ **Pipeline extension · Validate before selection**

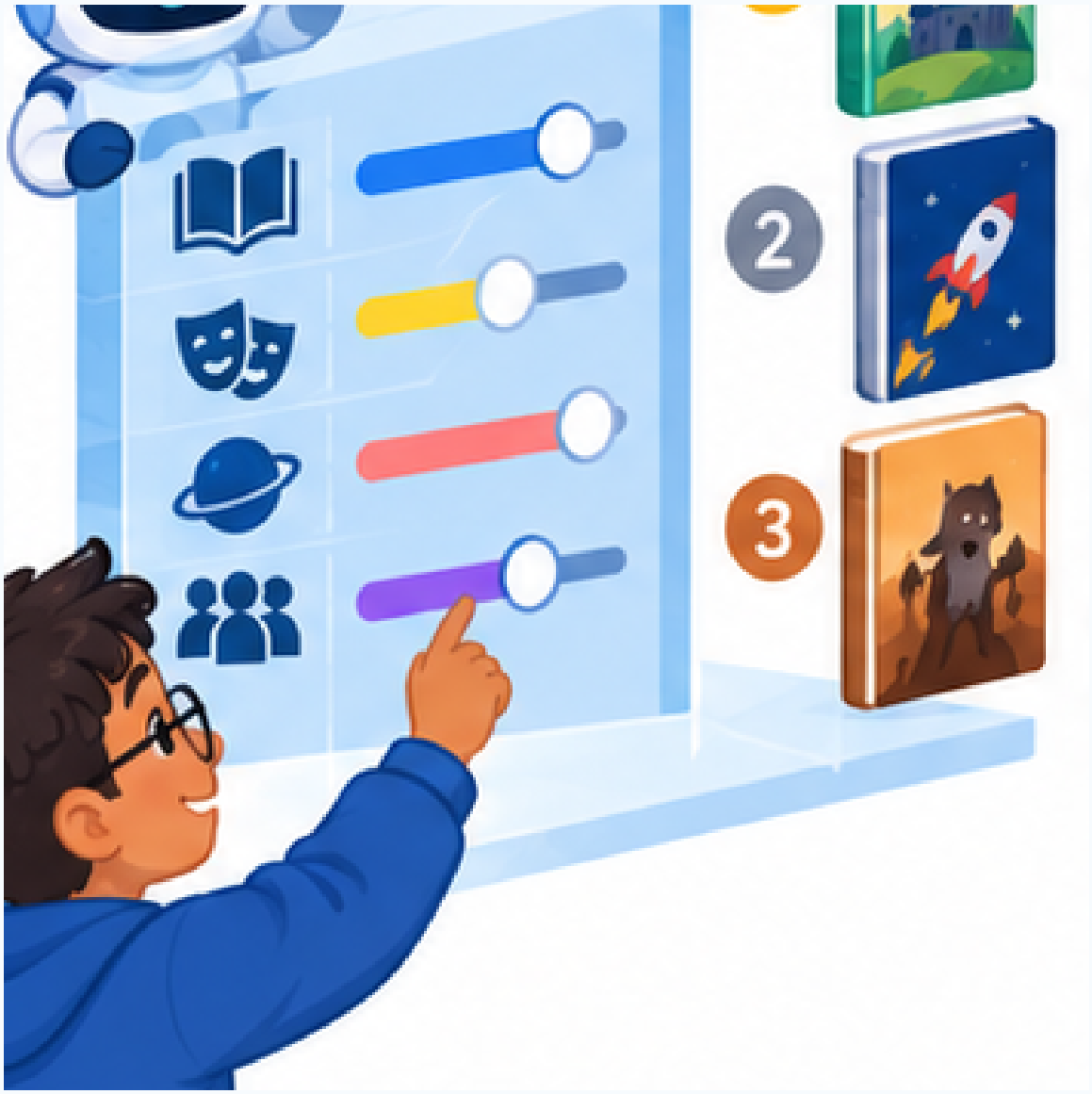
Build the validation **stage** of the planned chooser first. A valid record has nonempty text name, integer minutes 0 through 60 and integer fun and learning ratings 0 through 5. The batch **stage** must separately check name uniqueness. Only accepted records may reach the selection function in this week’s lesson.

▼ **Earlier wording, alternatives and picture transcripts**

PICTURE STORY

Notice → Predict → Explain

Begin with visible **evidence**. Do not explain more than the picture and information support.



MYSTERY

The learners must use build the **recommender** to solve a problem. What information do they have? What decision or calculation do they make? What would count as **evidence** that it worked?

Three **observation**s (what you can point to):

Write or draw your reasoning and evidence.

My **prediction** and the clue supporting it:

Write or draw your reasoning and evidence.

A closer explanation

Apply the connected idea: Building the data pipeline

MAIN LESSON · SECTION 3

Trace the best-so-far item through complete code

> The compact selector assumes its inputs have already passed validation.

Predict and run the complete activity example

```
def recommend(items, limit):
    best = None
    best_score = None
    for item in items:
        if item["minutes"] <= limit:
            score = 2 * item["fun"] + item["learning"]
            if (best is None or score > best_score
                or (score == best_score and item["name"] < best)):
                best = item["name"]
```

```
                best_score = score
    return best

items = [{"name": "B", "fun": 3, "learning": 2, "minutes": 10},
         {"name": "A", "fun": 2, "learning": 4, "minutes": 9}]
print(recommend(items, 10))
```

Trace equal scores and the name tie

Follow the evidence

STEP OR CASE	EVIDENCE	MEANING
B	Eligible score 8	Temporary best B
A	Eligible score 8	Alphabetical tie gives A

Read each row from left to right and connect the evidence to the stated rule.

The function prints nothing; the caller prints A. best starts as None to mean no eligible option found. Python short-circuits or: when best is None, later comparisons in that condition need not run.

After selecting an item, best_score is numeric. An empty list or no eligible item returns None.

Write a trace table with item, eligible, score and best-so-far. In the worked list, B qualifies and scores 8, so B becomes the current best. A also qualifies and scores 8.

The scores tie, so the written alphabetical rule changes the best to A. Starting with A gives the same final answer. If no item qualifies, best stays None; do not invent a winner or quietly relax the limit.

Explain the deciding step

- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ Pipeline extension · Complete single-row validator

```
def valid_item(row):
    if type(row) is not dict:
        return False
    if type(row.get("name")) is not str or not row["name"]:
        return False
    for key, maximum in [("minutes", 60), ("fun", 5), ("learning", 5)]:
        value = row.get(key)
        if type(value) is not int or not 0 <= value <= maximum:
            return False
    return True

print(valid_item({"name": "A", "minutes": 20, "fun": 4, "learning": 2}))
print(valid_item({"name": "B", "minutes": 10, "fun": "4", "learning": 2}))
```

Follow the evidence

STEP OR CASE	EVIDENCE	MEANING
Raw rows	Check fields and types	Accepted or rejected
Accepted rows	Check unique names	No ambiguous identifiers
Unique valid rows	Apply this week's selector	Suggestion plus counts

Read each row from left to right and connect the evidence to the stated rule.

The worked calls print True then False. Short-circuit checks keep absent or wrongly typed fields away from later operations.

This Boolean helper does not explain the reason; add reason codes in the reporting **stage** or use a validation result record. Do not describe a range failure as a no-match outcome.

Think of validation and selection as two desks. At the first desk, each record is checked for required fields, exact types and allowed ranges; a batch check also checks unique names.

Only accepted records go to the second desk, where the time limit and scoring policy are applied. A record can be valid yet too long to qualify.

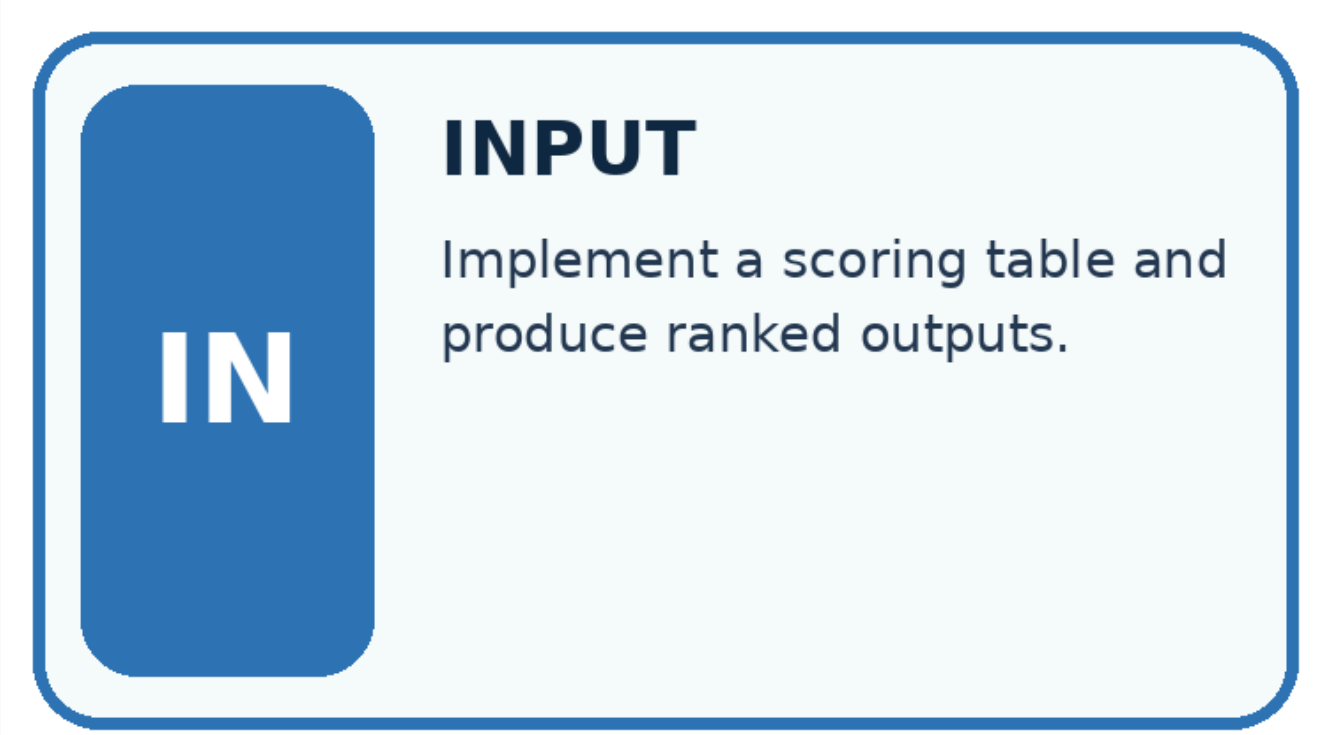
Report rejected and ineligible separately so a reader knows which decision removed an item.

▼ Earlier wording, alternatives and picture transcripts

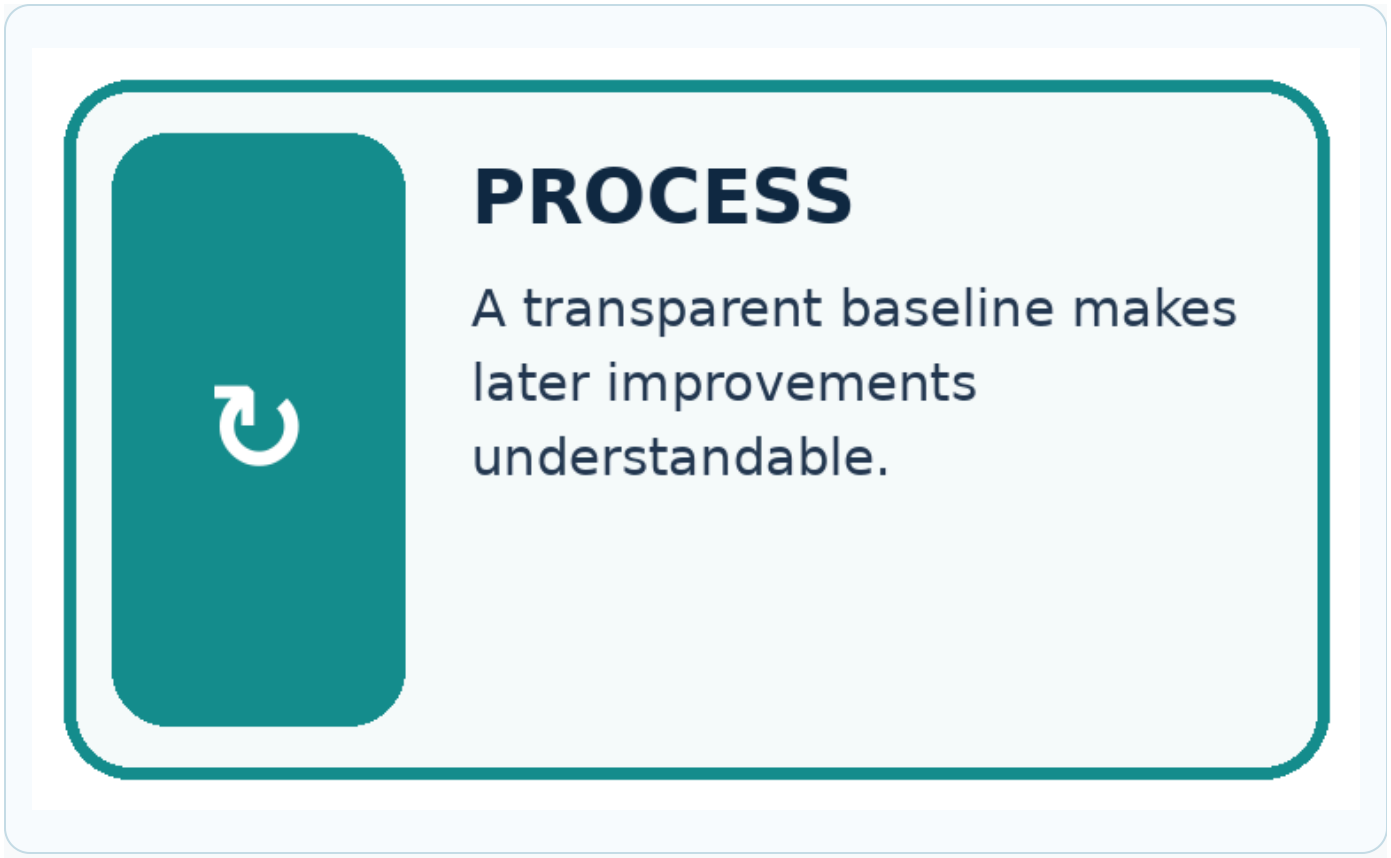
VISUAL **MECHANISM**

Input → **Process** → **Output**

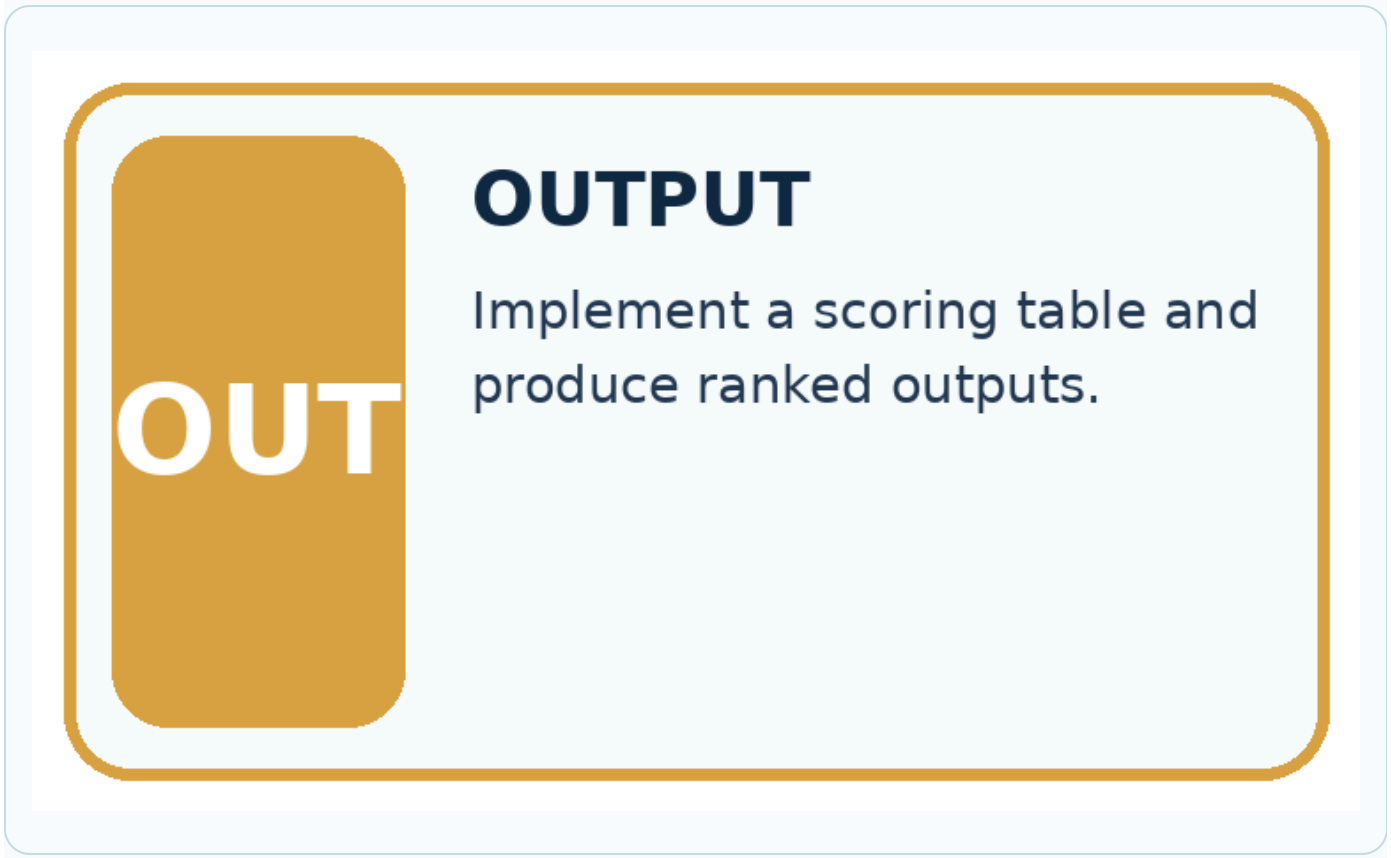
Trace the information. At each **stage**, ask what changed and how you could test it.



↓


↻
PROCESS
A transparent baseline makes later improvements understandable.

↓


OUT
OUTPUT
Implement a scoring table and produce ranked outputs.

Which **stage** could fail even when the other two are correct? Explain:

> **Write or draw your reasoning and evidence.**

A closer explanation

Trace the best-so-far record

Apply the connected idea: Building the data pipeline

Trace the hand-off between stages

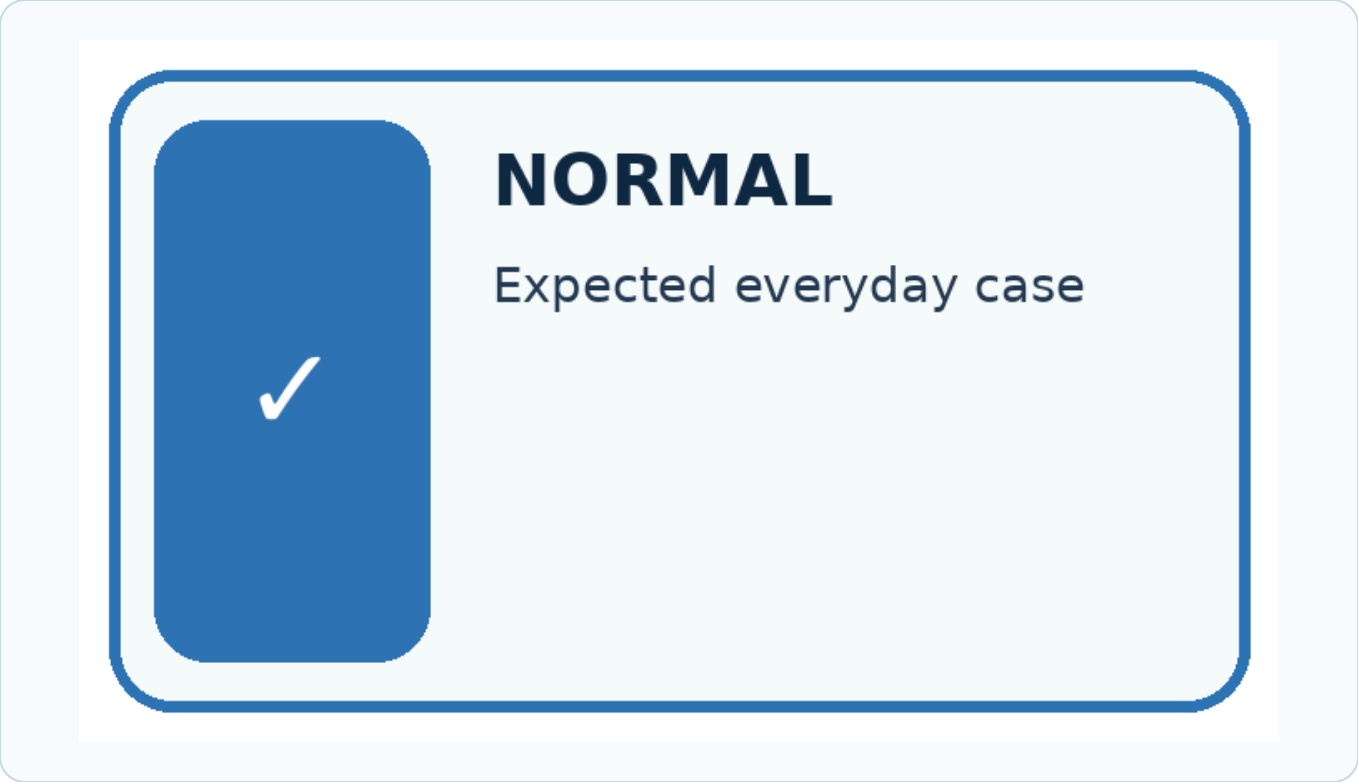
Connect the picture and the explanation

MAIN LESSON · SECTION 4

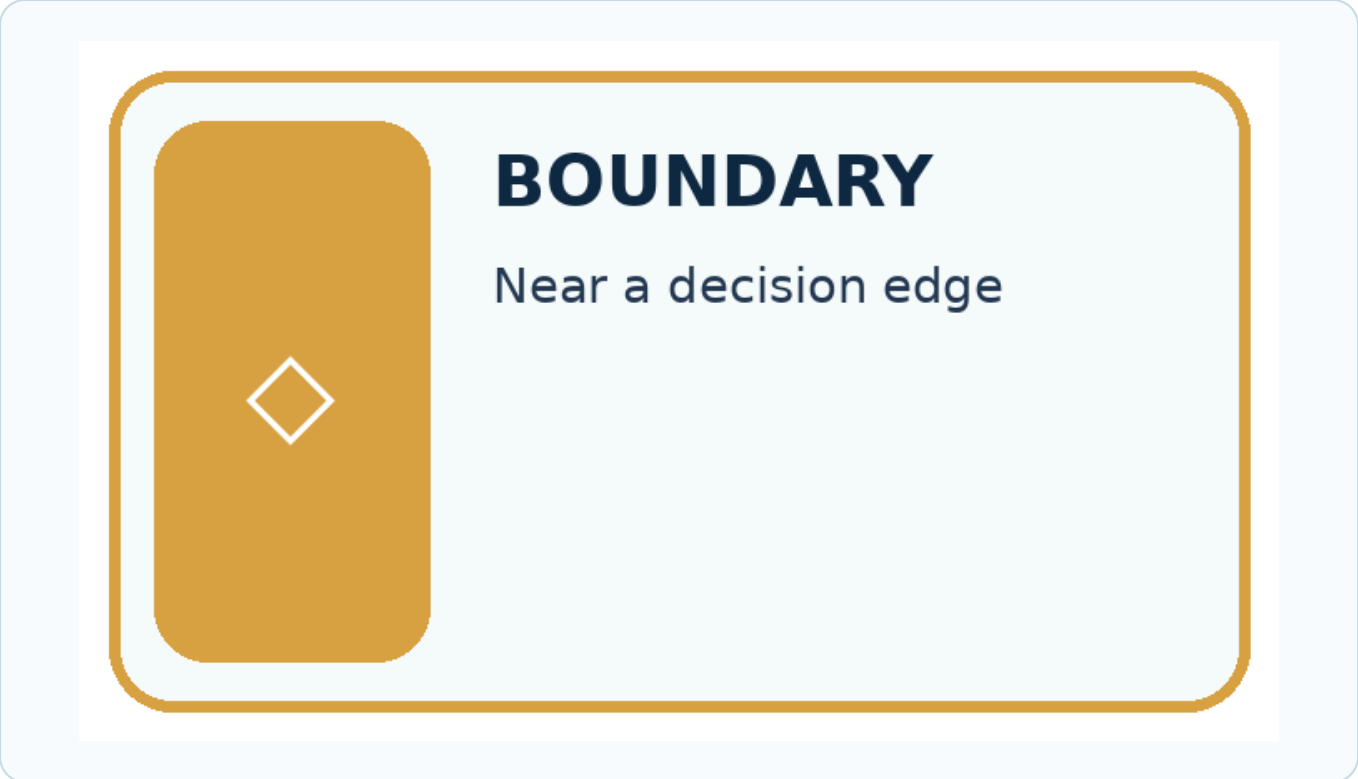
Compare eligibility, ranking and no-match behavior

> A high score cannot repair a failed hard constraint.

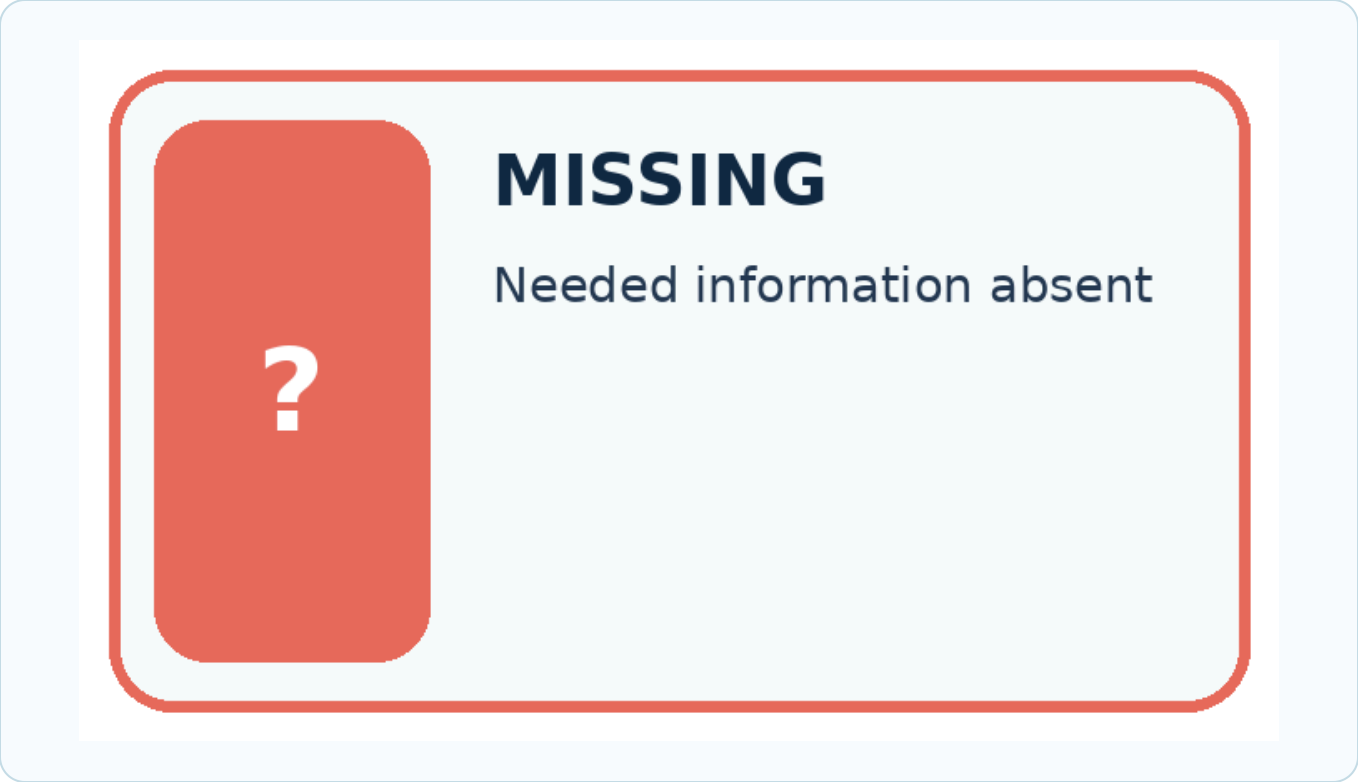
Four pictures with a purpose



Normal case: predict the result and deciding evidence.



Boundary case: identify the exact limit.



Missing information: name what must be clarified.



Transfer case: explain which procedure still applies.

Choose a revealing case

- Which case is most likely to reveal a hidden assumption? Why?

> **Write or draw your reasoning and evidence.**

Predict the supplied changed case

- Change: Change D’s pages to 91, leaving every other field unchanged.

▼ Check after predicting

Prediction: D becomes ineligible, so A wins with score 2.

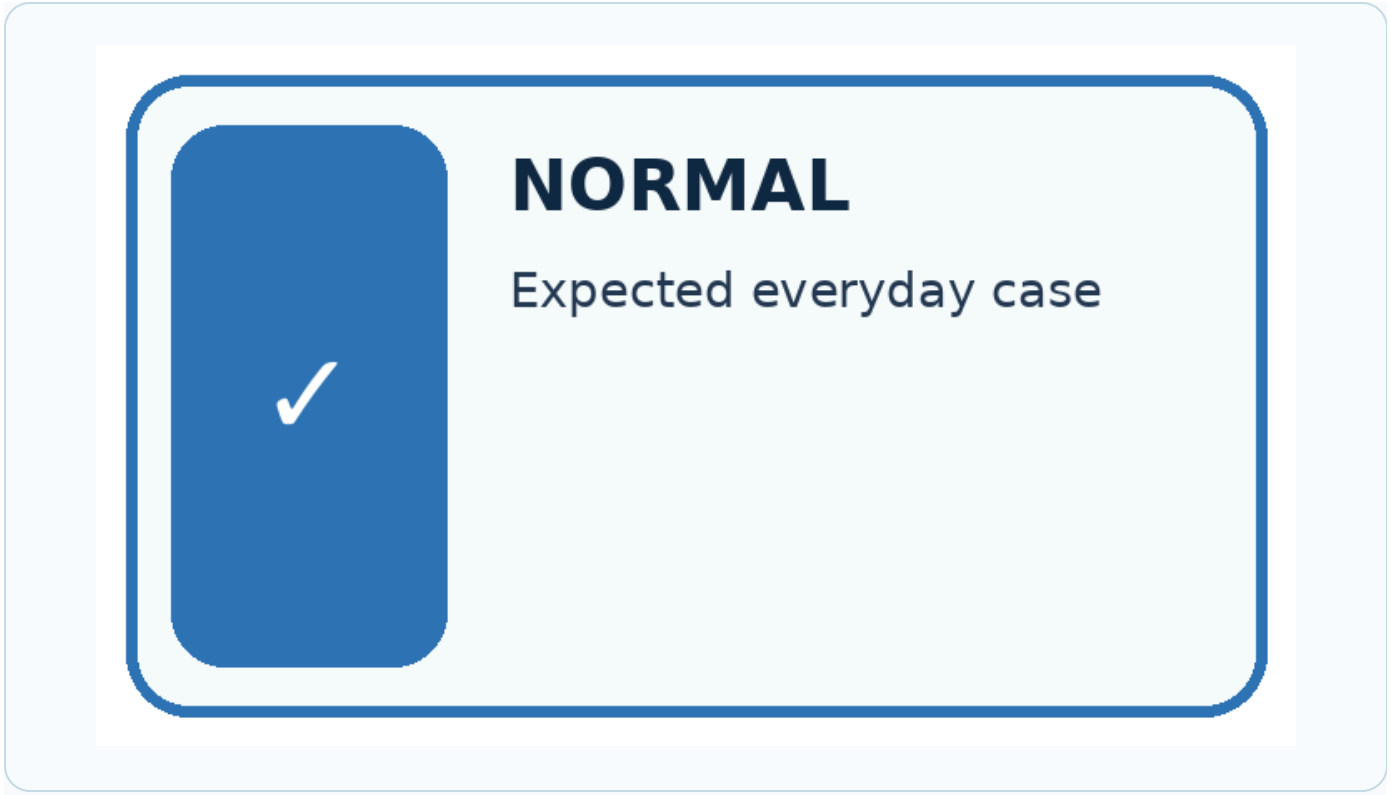
Reason: The strict excess over the 90-page maximum changes filtering before ranking.

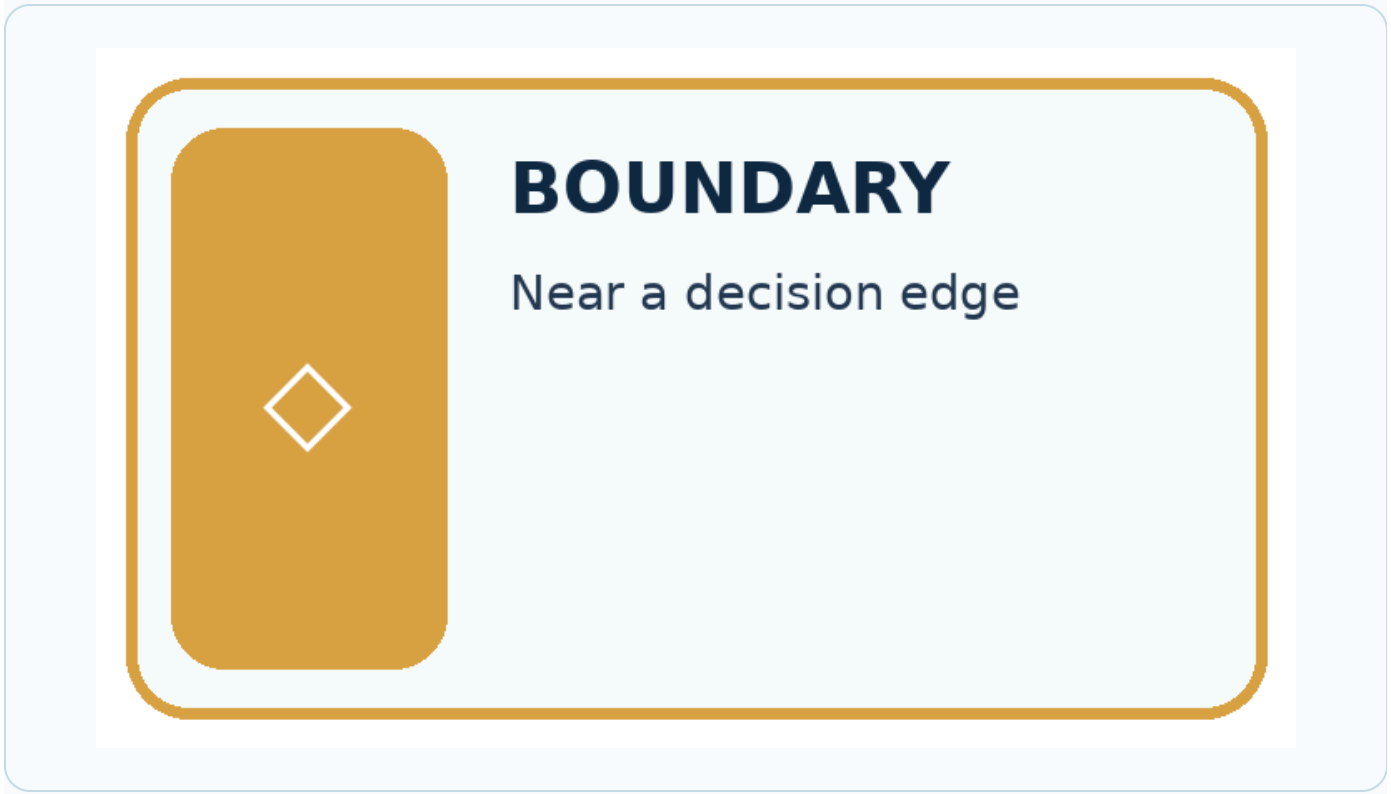
▼ Earlier wording, alternatives and picture transcripts

CASE GALLERY

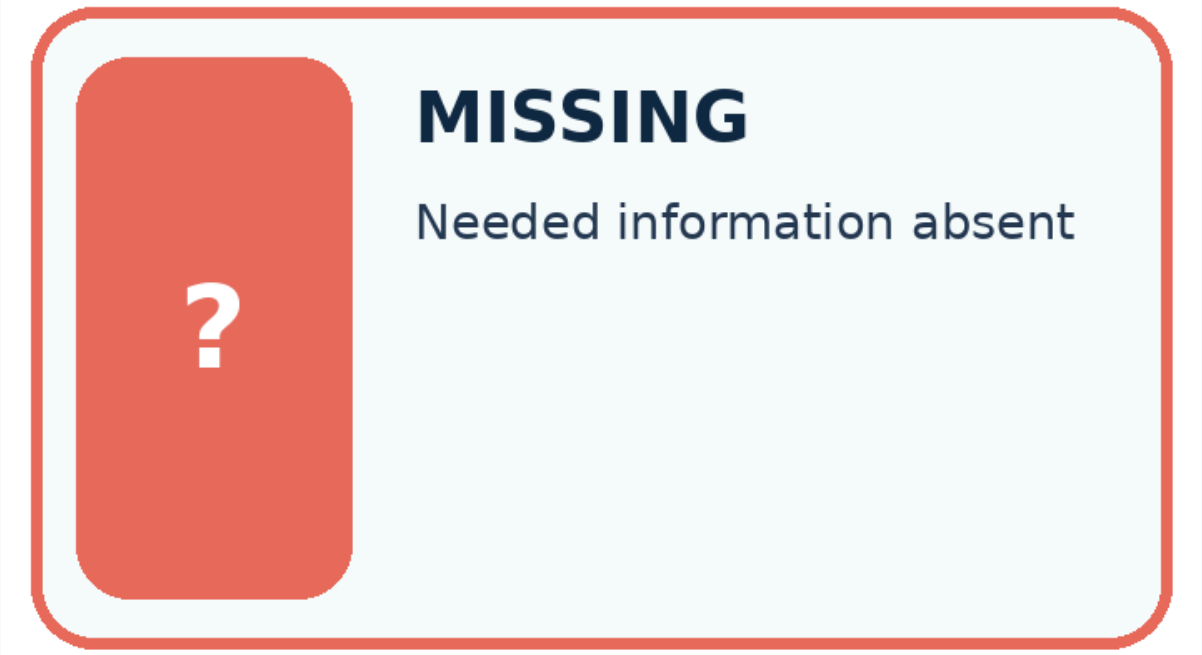
Four Tests, Four Kinds of **Evidence**

A system is not understood until it survives more than one comfortable example.





NORMAL CASE: predict and name **evidence**. BOUNDARY CASE: predict and name **evidence**.





MISSING INFORMATION: predict and name **evidence**. TRANSFER CASE: predict and name **evidence**.

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Test the activity selector, then a separate book ranking

> The book example uses illustrated/comic scores and ID ties, not activity fields.

Predict activity order reversal and limit8

Predict the result if the list order is reversed, and if the limit is 8.

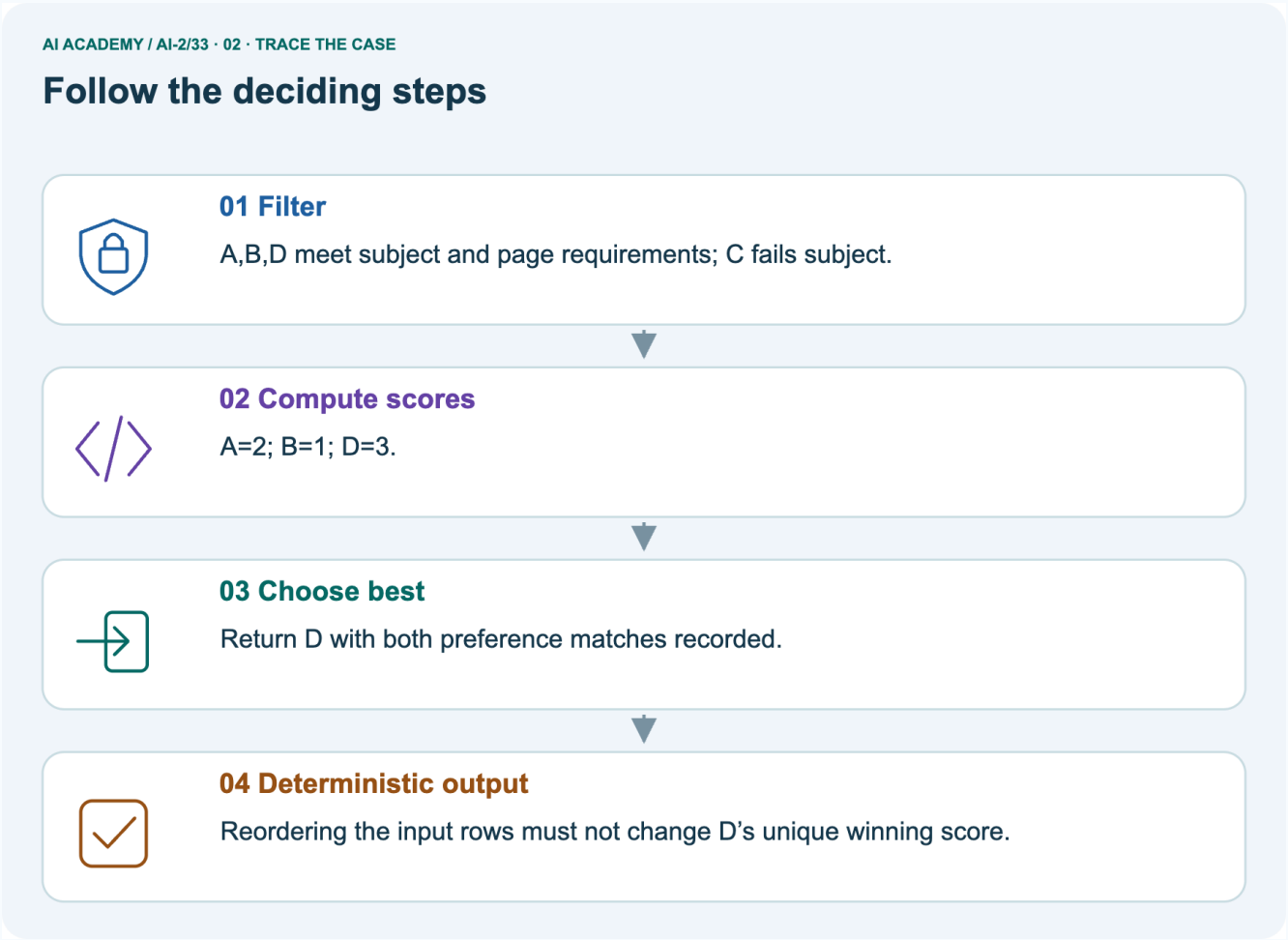
Read the separate animal-book request and rows

A fictional book chooser filters a request before comparing preference scores.

Request: animal subject, at most 90 pages. Score 2 for illustrated and 1 for comic. Tie chooses smaller ID.

A animal90 illustrated prose; B animal70 plain comic; C space60 illustrated comic; D animal80 illustrated comic.

Trace eligibility before preference scores



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Filter	A,B,D meet subject and page requirements; C fails subject.
2. Compute scores	A=2; B=1; D=3.
3. Choose best	Return D with both preference matches recorded.
4. Deterministic output	Reordering the input rows must not change D’s unique winning score.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Eligibility and ranking answer different questions. C has a high preference score but is not an animal book, so it must stay excluded.

The implementation needs a no-match result and explicit tie policy even when this example has a unique winner. Save rejected reasons for an **explanation**, not just the final ID.

Predict D at91pages

- Change one thing: Change D’s pages to 91, leaving every other field unchanged. Predict the result before checking the **explanation**. What remains the same?

▼ Check after predicting

AI ACADEMY / AI-2/33 · 04 · COMPARE THE CASES

Change one thing. Trace what follows.

Original case

Reordering the input rows must not change D's unique winning score.

Changed condition

Change D's pages to 91, leaving every other field unchanged.

Worked changed-case result

D becomes ineligible, so A wins with score 2.
Why: The strict excess over the 90-page maximum changes filtering before ranking.

Follow the labelled picture with the matching explanation. Open the vector diagram

Run the complete pictured book ranking

```
def choose_book(rows, subject, max_pages):
    # Inputs are supplied, validated fictional rows with unique IDs.
    eligible = [r for r in rows if r['subject'] == subject and r['pages'] <= max_pages]
    ranked = sorted(eligible, key=lambda r: (-(2*int(r['illustrated']) + int(r['comic'])),
r['id']))
    return [(r['id'], 2*int(r['illustrated']) + int(r['comic'])) for r in ranked]

books = [dict(id='A',subject='animal',pages=90,illustrated=True,comic=False),
         dict(id='B',subject='animal',pages=70,illustrated=False,comic=True),
         dict(id='C',subject='space',pages=60,illustrated=True,comic=True),
         dict(id='D',subject='animal',pages=80,illustrated=True,comic=True)]
print(choose_book(books, 'animal', 90))
changed = [dict(r, pages=91) if r['id']=='D' else dict(r) for r in books]
print(choose_book(changed, 'animal', 90))
print(choose_book(books, 'animal', 50))
```

Explain why fewer pages alone does not win

- Toy preference weights do not imply shorter or illustrated books are universally better for all learners.

- Read the labelled picture: Explain why B does not win after D is excluded even though it has fewer pages.

▼ Pipeline extension · Predict valid boundary fields

> **Predict validity for minutes 0, fun 0, learning 5 with name A, then for minutes 61.**

▼ Earlier wording, alternatives and picture transcripts

WORKED **EVIDENCE**

Reason Step by Step

Predict first. Then check each step. A correct answer without visible reasoning is incomplete **evidence**.

WEEK 33 • WORKED EVIDENCE

1. Weights: genre=3, difficulty=2, length=1.
2. Compute every candidate’s terms and total.
3. Filter hard constraints, sort remaining totals high to low.
4. Output top choice, score breakdown, one limitation, and a tie rule.

1. Weights: genre=3, difficulty=2, length=1.

2. Compute every **candidate**’s terms and total.

3. Filter hard constraints, sort remaining totals high to low.

4. **Output** top choice, score breakdown, one limitation, and a tie rule.

Recalculate, retrace, or restate the reasoning in your own representation:

> **Write or draw your reasoning and evidence.**

CHECK Did the **evidence** answer the exact question? Did the total, path, or source record stay consistent?

A closer explanation

Apply the connected idea: Building the data pipeline

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Build the Recommender

The facts we will use

AI ACADEMY / AI-2/33

Filtering before scoring

	Pages	Preference score	Status
A	90	2	eligible
B	70	1	eligible
C	60	3	subject fails
D	80	3	eligible

Read a cell using both its row and column labels.

Follow the labelled picture with the matching explanation. Open the vector diagram

AI ACADEMY / AI-2/33

Build the Recommender

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.

1. Filter

A,B,D meet subject and page requirements; C fails subject.

2. Compute scores

A=2; B=1; D=3.

3. Choose best

Return D with both preference matches recorded.

4. Deterministic output

Reordering the input rows must not change D's unique winning score.

CHANGE ONE THING

Change D's pages to 91, leaving every other field unchanged.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Repair an order-dependent tie

> Use the declared policy rather than whichever row happens to be last.

Find the first policy mismatch

Claim to repair: The last tied activity should replace the current best. In the main activity contract, equal scores use ascending name, so A wins over B in either order. Compare that with the separate independent task's shorter-page tie rule only when you reach that task. Do not silently substitute one policy for another. Test reversed order and no eligible items after repairing order dependence.

Propose a case that would reveal it

> **Write a one-sentence explanation of X winning without calling it objectively best.**

▼ **Earlier wording, alternatives and picture transcripts**

ERROR DETECTIVE

Compare → Diagnose → Repair

Do not patch the answer. Find the earliest unsupported assumption, wrong step, or weak **evidence**.

!

COMPARE • DIAGNOSE

Weak idea: A recommendation is transparent if it displays only the final score.
Strong move: locate evidence, change one cause, and retest.

Evidence record	Expected	Actual	Likely cause / next test
normal case	works		
boundary case	stated rule		
missing input	safe response		
fresh transfer	generalizes		

Repair one cause. Why is this repair better than changing several things?

Write or draw your reasoning and evidence.

A closer explanation

The recommendation follows a written scoring policy. It does not learn preferences, prove learning benefit or authorize an action. Display the chosen rule and let a person review whether an option is suitable.

Apply the connected idea: Building the data pipeline

Each **stage** should have a clear **input** and **output** contract. A program can be correct at one **stage** but wrong at their connection, such as passing rejected rows into selection. Keep acceptance and rejection counts in the report.

> **Two valid rows have the same name but different ratings. Should individual validation alone permit selection?**

MAIN LESSON · SECTION 8

Practise a no-match trace with less help

> A schema rejection, an ineligible item and no eligible result are different outcomes.

Repair an incorrect tied update

Updating best on score >= best_score makes the last tied item win, so reordering can change results. Use the explicit name comparison for ties. Test both orders and a no-match case after the repair.

Complete main W2 using the original activity list

Practise with less help: Handle no match

Use workbook W2 for the connected example “Implementing a **recommender**”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Repair an overclaim before discussion

- Use the worked case for Implementing a **recommender**. Proposed **explanation**: “A successful run proves the conclusion”. Decide whether this **explanation** is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Check your reasoning

Explain the separate pictured ranking

- Explain why B does not win after D is excluded even though it has fewer pages.

Explain the picture

> **My explanation using a pictured step and the given facts**

Keep independent work separate

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ Pipeline extension · Send only accepted rows to selection

Calling the selector on raw data can crash or produce a result from invalid ratings. Route data through validation first. Duplicate names require a batch check even when each row individually passes; exclude every occurrence of each duplicated name, retain reasons and review the source instead of arbitrarily keeping one.

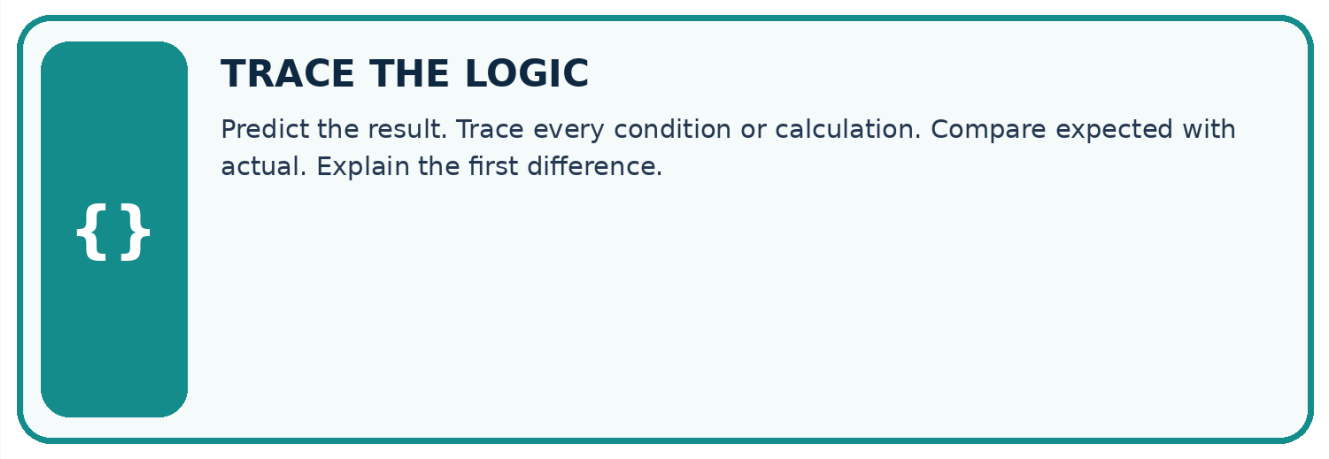
Use its named workbook W2. Identify the given **input**s, try the middle steps with less help, then explain your trace. Check this claim before group discussion: A successful run proves the conclusion

▼ Earlier wording, alternatives and picture transcripts

LOGIC STUDIO

Trace Before You Run

Block code, pseudocode, tables, and diagrams are different ways to expose the same reasoning.



SET **evidence**_record PREDICT expected_result RUN one normal case and one boundary case COMPARE actual_result with expected_result IF different: locate first cause; change one factor; RETEST EXPLAIN result, limitation, and next safe step

Trace one **input**. Which line changes the state or chooses a path?

> **Write or draw your reasoning and evidence.**

A closer explanation

Apply the connected idea: Building the data pipeline

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own **explanation**.

Practise the connected topic: Building the data pipeline

MAIN LESSON · SECTION 9

Run a complete validation-to-ranking pipeline

> Inspect accepted, rejected, ineligible and ranked records separately.

The complete standalone implementation

```
from collections import Counter

def pipeline(raw, limit):
    if type(limit) is not int or not 0 <= limit <= 60:
        return {'error': 'invalid request'}
    names = Counter(r['name'] for r in raw if type(r) is dict
                    and type(r.get('name')) is str and r['name'])
    accepted, rejected = [], []
    for index, row in enumerate(raw, 1):
        reason = None
        if type(row) is not dict:
            reason = 'not a record'
        elif type(row.get('name')) is not str or not row['name']:
            reason = 'invalid name'
        elif names[row['name']] > 1:
            reason = 'duplicate name'
```

```
        else:
            for key, maximum in [('minutes',60),('fun',5),('learning',5)]:
                value = row.get(key)
                if type(value) is not int or not 0 <= value <= maximum:
                    reason = 'invalid ' + key
                    break
        if reason:
            rejected.append((index, reason))
        else:
            accepted.append(dict(row))
    eligible = [r for r in accepted if r['minutes'] <= limit]
    ranked = sorted(eligible, key=lambda r: (-(2*r['fun']+r['learning']),r['name']))
    scores = [(r['name'],2*r['fun']+r['learning']) for r in ranked]
    return {'winner': scores[0][0] if scores else None, 'ranked': scores,
            'accepted': len(accepted), 'rejected': rejected,
            'ineligible': [r['name'] for r in accepted if r['minutes']>limit]}

raw = [dict(name='A',minutes=20,fun=3,learning=2),
       dict(name='B',minutes=21,fun=5,learning=5),
       dict(name='C',minutes=12,fun=2,learning=4),
       dict(name='D',minutes=10,fun='4',learning=2)]
print(pipeline(raw,20))
print(pipeline(list(reversed(raw)),20)['ranked'])
print(pipeline(raw,0)['winner'])
print(pipeline(raw,True))
```

Predict the supplied results

Predict before running: A and C score8 and qualify at20minutes; B is valid but too long; D fails the fun type. Expected accepted3of4, rejected row4, ineligibleB, ranked[(A,8),(C,8)], winnerA. Reverse the input to check the same ranking; rejection row numbers then refer to the new input order. Limit0 produces no eligible option; Boolean True is an invalid request, not a1-minute request.

Test boundaries and preserve evidence

1. Use this complete local pipeline for the investigation. Compare each output with your frozen Week32 expectations. Add duplicate copies of A: every A occurrence must be rejected under the planned duplicate policy, with C still eligible. Add a missing field, a Boolean rating, an empty list and a duration exactly20. Preserve original rows and rejection reasons. For the separate book example, run the supplied full ranking and changed-D case. Do not describe a written score rule as a trained model or its outputs as real learner preferences. Reserve the mystery-book independent task for its first attempt.

Keep expected and actual results

Test / input · Prediction · Actual · Evidence / calculation · Change or keep?

> **Test / input:** normal
>
> **Prediction:** _____
>
> **Actual:** _____
>
> **Evidence / calculation:** _____
>
> **Change or keep?:** _____

> **Test / input:** boundary
>
> **Prediction:** _____
>
> **Actual:** _____
>
> **Evidence / calculation:** _____
>
> **Change or keep?:** _____

> **Test / input:** missing
>
> **Prediction:** _____
>
> **Actual:** _____
>
> **Evidence / calculation:** _____
>
> **Change or keep?:** _____

> **Test / input:** fresh transfer
>
> **Prediction:** _____
>
> **Actual:** _____
>
> **Evidence / calculation:** _____
>
> **Change or keep?:** _____

What did the hardest test teach you that the normal test did not?

> **Write or draw your reasoning and evidence.**

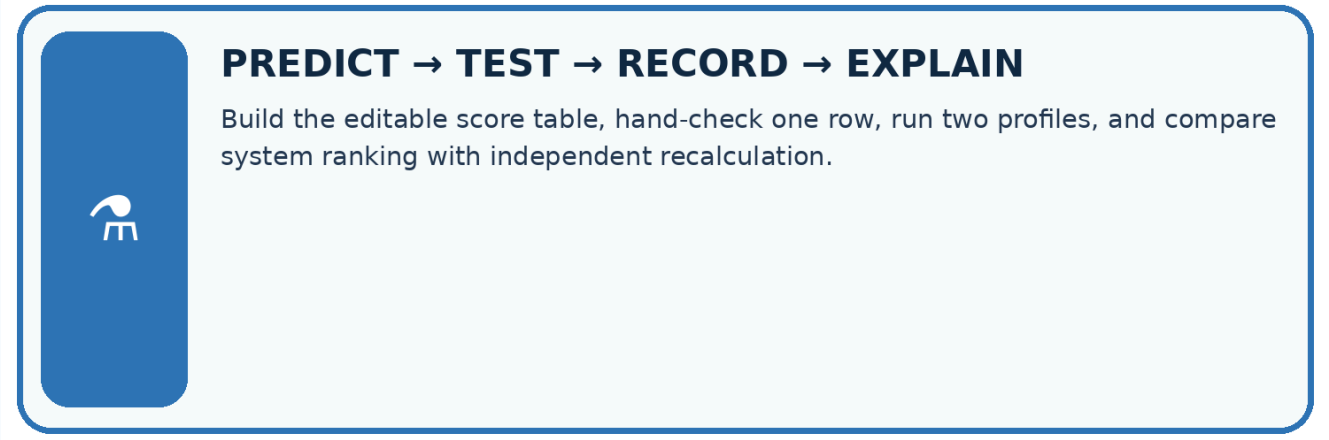
Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ **Earlier wording, alternatives and picture transcripts**

EVIDENCE LAB

Predict → Test → Record → Explain

Keep expected and actual results separate. Surprises are useful data when recorded honestly.



Run the supplied complete activity pipeline and pictured book ranking on fictional local records. Hand-check scores, reverse order, test duplicates and invalid fields, and compare a valid no-match result with an invalid request. No live learner profile is required.

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: rank the fresh mystery-book catalogue

> Filter first and follow this task's own stated tie policy.

The independent records and policy

Request: mystery, at most 100 pages. Catalogue: D mystery110 illustrated; E mystery80 plain; F science80 illustrated; G mystery95 illustrated; H mystery90 illustrated.

Filter first; score 3 for mystery plus 2 for illustrations; break ties by fewer pages.

This mystery-book assessment has its own score and tie policy: use the supplied mystery/page requirements,3genre+2illustration points and shorter-page tie. Do not reuse the worked activity name tie or animal-book ID tie. The supplied rows are fictional and complete for this comparison; no live lookup or new preference estimate is required.

Your first attempt and help record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Your teacher has the independent answer and scoring criteria. Complete the matching workbook week to keep your practice **evidence** together.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ **Earlier wording, alternatives and picture transcripts**

Independent case: Changed independent case

Week 33 Independent application

Close the worked **explanation**. Apply the idea to this different question without copying.

A reader requests space and at most 60 pages. Book A is space, 90 pages; B is animals, 40 pages. What should the prototype return if both conditions are required?

My answer, with a trace, calculation or supporting **evidence**

Write or draw your reasoning and evidence.

Create and test

Change one relevant detail. Predict the result before testing. Include a boundary or missing-information case when it helps reveal a limit.

My changed **input** and expected result

> **Write or draw your reasoning and evidence.**

Connect to your project

This is the complete **baseline**: **input**s, scoring, filter, ranking, **explanation**, and abstain rule.

The **evidence** I will keep

> **Write or draw your reasoning and evidence.**

Transfer practice from the original programme

Catalogue: A mystery/90/illustrated, B mystery/140/not illustrated, C science/80/illustrated. Request mystery, maximum 100 pages. Rank with 2 genre points and 1 illustration point after filtering.

Show why other books are excluded.

> **Explain the mechanism, evidence and one limit.**

Use feedback to improve

MAIN LESSON · SECTION 11

Save a baseline that can be inspected and rerun

> Keep the full trace beside the selected result.

This week’s concrete contribution

Keep the implemented baseline with its schema and versioned score/tie contract, complete ranked output, exclusion reasons and acceptance results. The activity pipeline and book chooser illustrate separate schemas; use the book contract for the recommender project. Show a no-eligible-option result without relaxing a hard constraint. Preserve the human decision owner and state that written preference weights are neither learned preferences nor proof of learning benefit.

Save the **recommender** code, contract and this full trace; test the same records in different **input** orders.

Keep the checkpoint and human choice

THIS WEEK’S CHECKPOINT This is the complete **baseline**: **input**s, scoring, filter, ranking, **explanation**, and abstain rule.

SYSTEM MAP Reader need → allowed **input**s → transparent rule or score → ranked **candidate**s → **explanation** → test record → **human decision**

What will you add, change, or verify in the project this week?

> **Write or draw your reasoning and evidence.**

What should remain under **human control**?

Write or draw your reasoning and evidence.

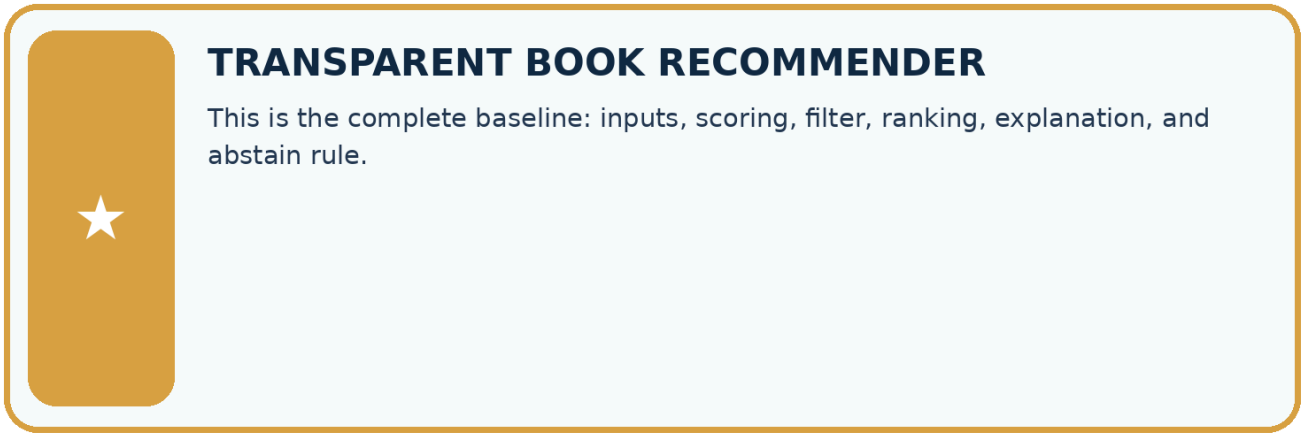
RESPONSIBLE DESIGN Collect only necessary information. Show uncertainty and limits.
Never confuse a ranking with a fact about a person.

▼ Earlier wording, alternatives and picture transcripts

CUMULATIVE PROJECT

Transparent Book **Recommender**

Every week adds one inspectable piece. The system must show how it reached a result.



Evidence for your project

Finish: explain eligibility, score and tie separately

> Show why the result follows from this contract and these inputs.

Explain without notes

- Close your notes. Explain this goal in your own words: Implement a **deterministic recommender**

> **My deciding step, evidence and one remaining question**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Words for the main lesson and extension**

REFERENCE · SECTION 5

Reference · Words for the main lesson and extension

> Use each word with a concrete record or calculation.

Core words

Word	My own example	Not this / boundary
baseline	A simple reference method used to judge whether a more complex method adds value.	
candidate	An item or proposed answer being considered before a final selection.	
weighted total	A sum formed after multiplying each contribution by its assigned weight.	
explanation	An account connecting a result to the mechanism, calculation or evidence that supports it.	

Main explanation terms

Word	Meaning
Recommender	A procedure that suggests options using a stated policy
Tie	Two options have the same comparison score

Word	Meaning
Deterministic	The same inputs and rules produce the same result

Extension terms

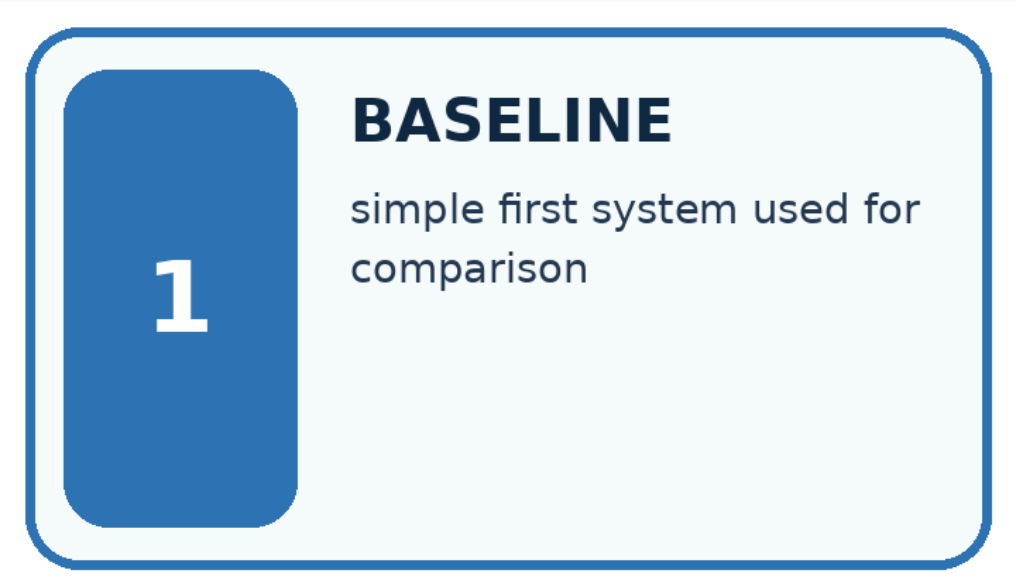
Term	Meaning
Stage	One part of a processing pipeline
Validation function	A function that decides whether an input meets a schema
Separation of concerns	Keeping different responsibilities in distinct parts
Rejection reason	An explanation of why a record was not accepted

▼ Earlier wording, alternatives and picture transcripts

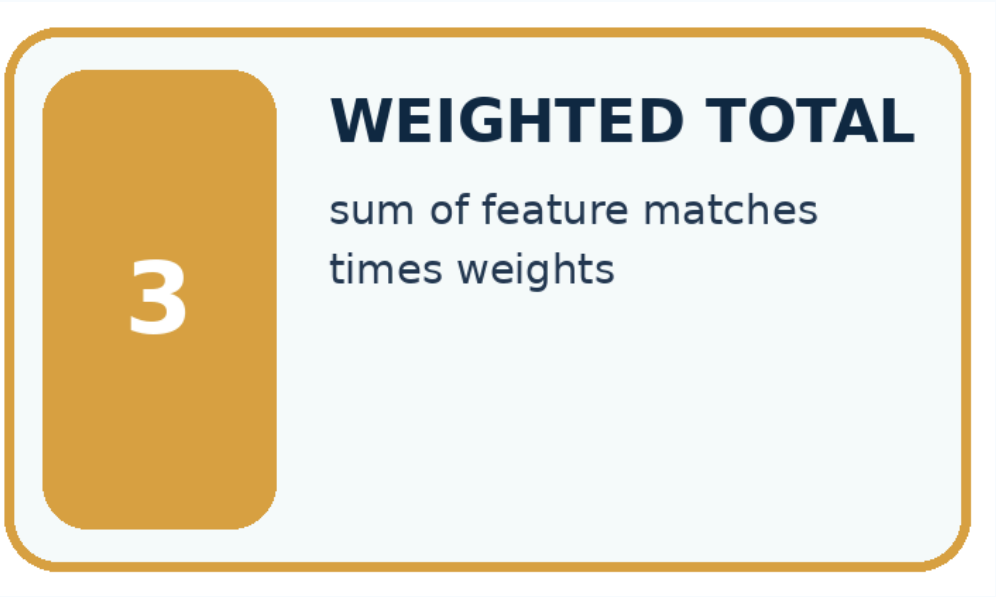
VOCABULARY LAB

Words You Can Use to Reason

A definition matters when you can recognize the idea, use it, and contrast it with a near-miss.




2
CANDIDATE
item being scored


3
WEIGHTED TOTAL
sum of feature matches
times weights



Words used in the closer explanation

Terms for Building the data pipeline

▼ **Extra challenge · Choose a stronger explanation**

REFERENCE · SECTION 10

Extra challenge · Choose a stronger explanation

> Choose the next useful challenge rather than repeating the ladder.

Thinking ladder

- 1 • NAME: Define the concept without repeating the textbook.
- 2 • TRACE: Show the information, state, branch, calculation, or **evidence** path.
- 3 • APPLY: Solve a different example independently.

- 4 • CHALLENGE: Find a boundary case, counterexample, or unfair outcome.
- 5 • EXPLAIN LIMITS: Say when the result should not be trusted or used alone.
- My strongest **evidence** of independent understanding:

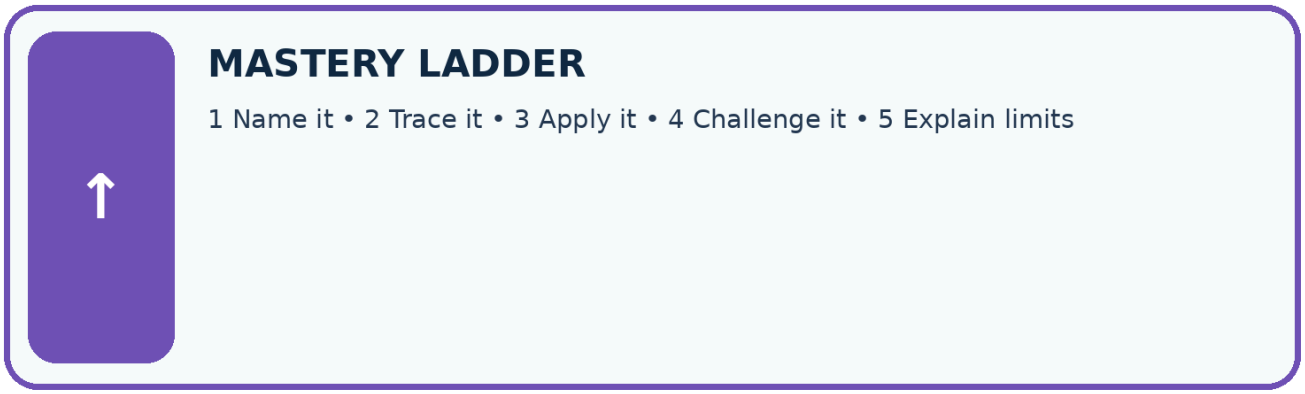
Write or draw your reasoning and evidence.

▼ Earlier wording, alternatives and picture transcripts

MASTERY

From Knowing the Word to Owning the Idea

Move upward only when you can produce **evidence** without copying the worked example.



▼ Reference · Check your explanation

Reference · Check your explanation

> Use after practice; Rank the new mystery catalogue independently is the selected independent task.

Evidence and limits

- 1. My fresh example, trace, calculation, or design:
- 2. My **evidence** that it works (include a boundary or missing-data case):
- 3. One limitation, fairness concern, or situation needing human review:

Write or draw your reasoning and evidence.

Write or draw your reasoning and evidence.

Write or draw your reasoning and evidence.

Self-check

SELF-CHECK ☐ I explained the **mechanism** ☐ I used **evidence** ☐ I tested a fresh case ☐ I named a limitation ☐ I can answer ‘why?’

▼ Earlier wording, alternatives and picture transcripts

INDEPENDENT MASTERY PROOF

Checklist for this week’s explanation

Use after practice. Rank the new mystery catalogue independently is the selected independent task.



Use the selected independent book task’s exact constraints, score and shorter-page tie.

▼ Extra practice · The earlier complete case

REFERENCE · SECTION 13

Extra practice · The earlier complete case

> Try the question before opening the worked explanation.

Question and given facts

A has flags (1,1), B=(1,0), C=(0,1); score=2g+l.

Compute the winner and **evidence** to display.

Your explanation

- Which detail supports the answer, and why does it matter?

> **Write or draw your reasoning and evidence.**

▼ Check after trying

A wins at 3; B=2; C=1. Show flags, formula, scores and **explanation**; declare tie policy.

Evidence or stage	Value or result
A, genre and length	3
B, genre only	2
C, length only	1

▼ Earlier wording, alternatives and picture transcripts

Week 33 Worked case

Use the supplied details to practise the weekly concept before attempting the independent question. This case extends the illustrated lesson.

The situation

The question

Worked explanation

Explain it in your own words

Trace the supplied case visually

Build and expose the scoring calculation

A has flags (1,1), B=(1,0), C=(0,1); score=2g+l.

Compute the winner and **evidence** to display.

A wins at 3; B=2; C=1. Show flags, formula, scores and **explanation**; declare tie policy.

▼ **Reference · Earlier checklist**

REFERENCE · SECTION 15

Reference · Earlier checklist

> Use when it helps inspect the evidence.

Checklist and first prediction

By the end, I can	How I will show it
Implement a deterministic recommender	Explain the deciding step, show a trace or test, and state one limit.
Handle ties and no eligible options	Explain the deciding step, show a trace or test, and state one limit.
Explain recommendations with their limits	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ **Earlier wording, alternatives and picture transcripts**

Start with this question: A fictional book chooser filters a request before comparing preference scores.

▼ **Extra practice · Pictures, feedback and repair**

REFERENCE · SECTION 17

Extra practice · Pictures, feedback and repair

> Try before opening feedback; keep the selected independent task separate.

First supported question

- A fictional book chooser filters a request before comparing preference scores.
- Which labelled step matches this detail? Reordering the **input** rows must not change D's unique winning score.
- 1. Choose best
- 2. Filter
- 3. **Deterministic output**
- 4. Compute scores
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the **stage** that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check after trying**

After your attempt, compare with the supported **explanation**: **Deterministic output**
Reordering the **input** rows must not change D's unique winning score. Eligibility and ranking answer different questions.

C has a high preference score but is not an animal book, so it must stay excluded. The implementation needs a no-match result and explicit tie policy even when this example has a unique winner.

Save rejected reasons for an **explanation**, not just the final ID.

Changed-case question

- Changed-case practice: Change D's pages to 91, leaving every other field unchanged. Which conclusion is supported by the supplied facts?
- 1. D becomes ineligible, so A wins with score 2.
- 2. This one example guarantees correct results for every future **input**.
- 3. We can decide the result without examining the changed **input** or the procedure.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check changed case**

After your attempt, compare with the supported **explanation**: D becomes ineligible, so A wins with score 2. The strict excess over the 90-page maximum changes filtering before ranking.

Repair and keep evidence

Review Project Problem and Success Test (ai-2/32). Goal: Define a small application purpose. Short practice: Write an equality test and explain which requirement it checks.. Return to this week and explain what you changed.

Review Agents and Tools (ai-2/30). Goal: Explain the **mechanism** and terminology of state machines for bounded agents. Short practice: Can the assistant accept Trace B because its final state is done?.

Return to this week and explain what you changed.

Review Predictive vs Generative (ai-2/27). Goal: Distinguish discriminative **prediction** from generation by inspecting the requested **output**.

Short practice: Does matching the physical box in the changed table prove the generator measures boxes?. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ Earlier wording, alternatives and picture transcripts

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

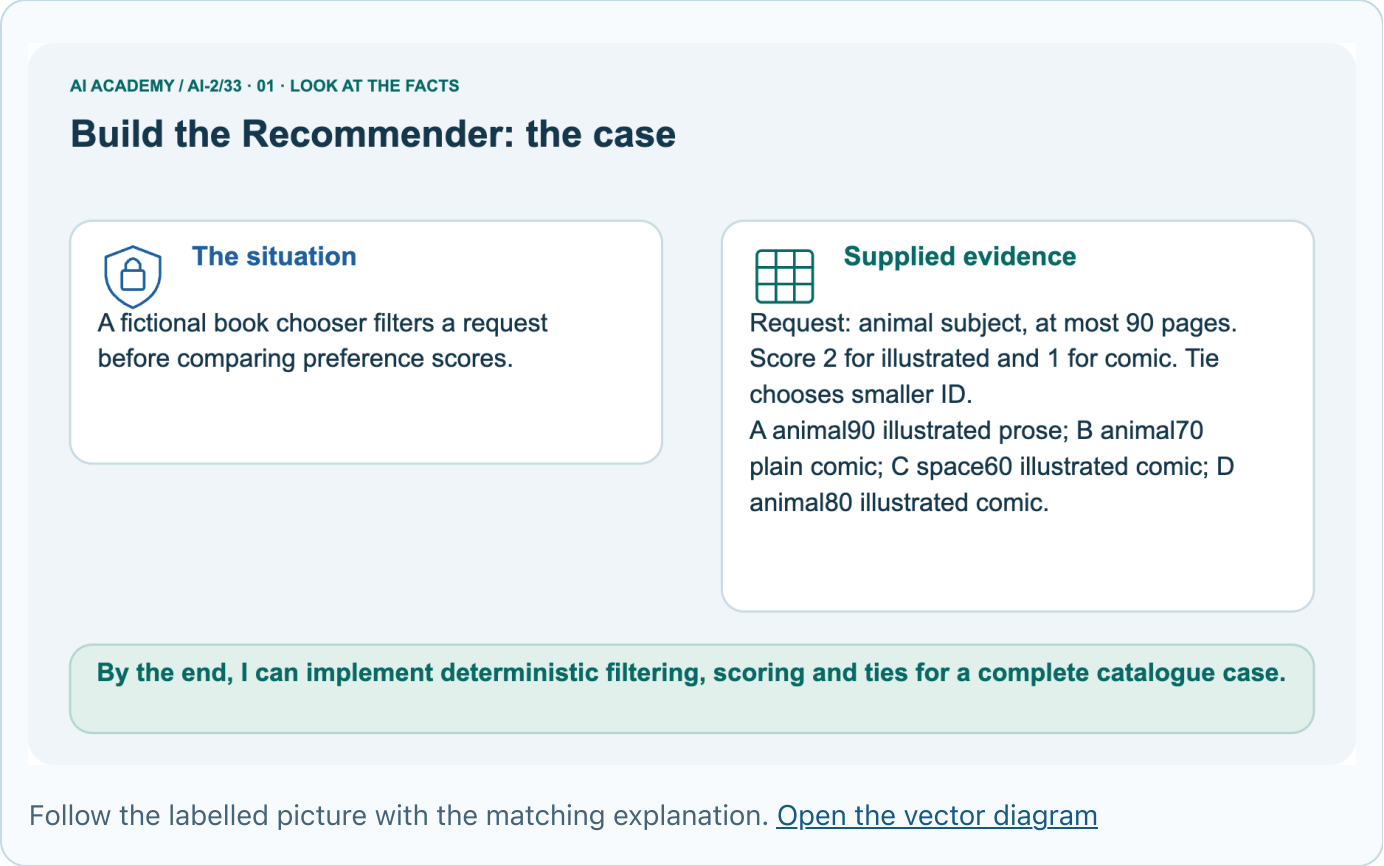
▼ Reference · The six-picture journey

REFERENCE · SECTION 18

Reference · The six-picture journey

> Revisit the picture that serves the current question.

01. Build the Recommender: the case



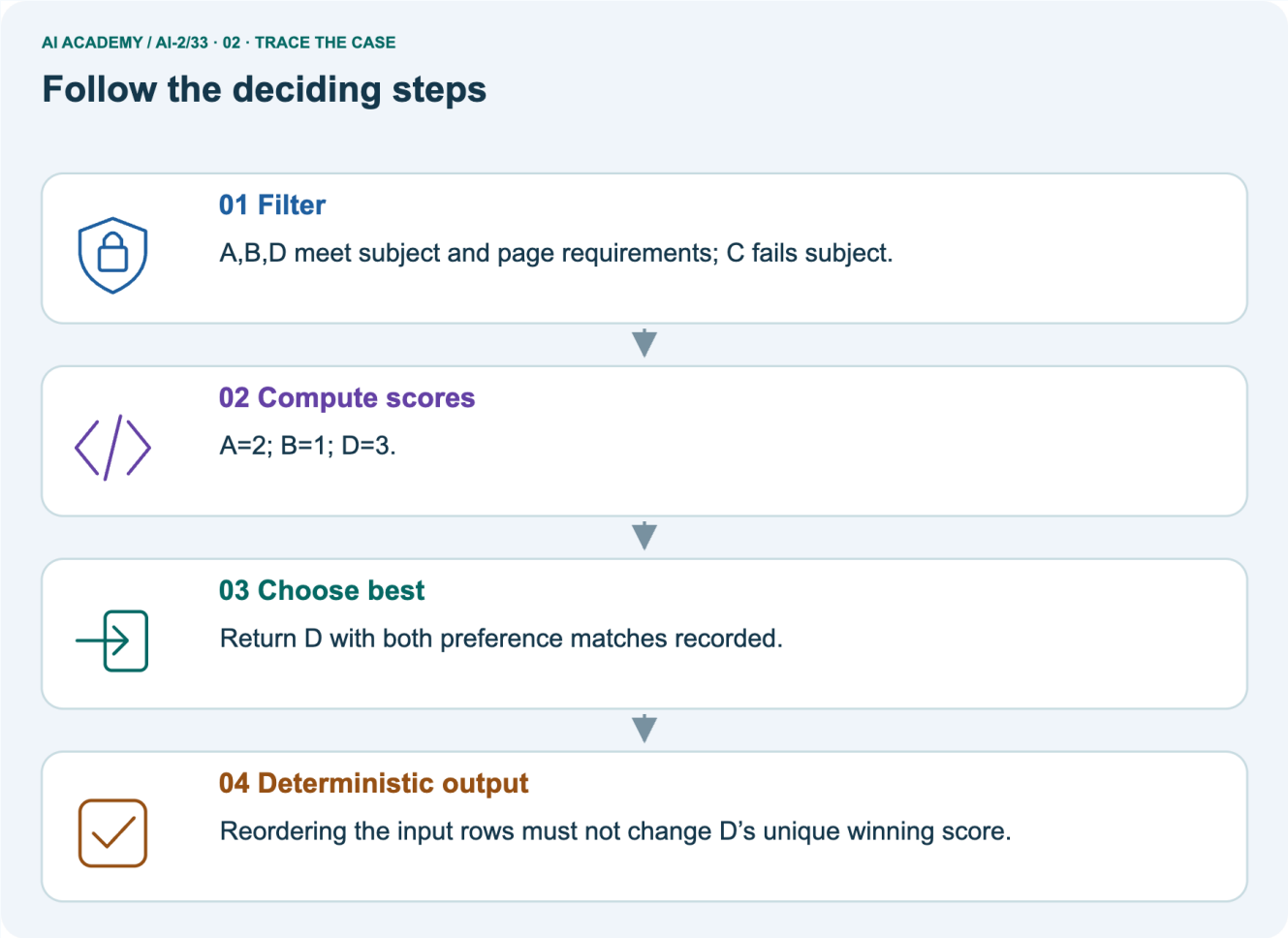
Follow the labelled picture with the matching explanation. Open the vector diagram

A fictional book chooser filters a request before comparing preference scores. Supplied facts: Request: animal subject, at most 90 pages. Score 2 for illustrated and 1 for comic. Tie chooses smaller ID.

A animal90 illustrated prose; B animal70 plain comic; C space60 illustrated comic; D animal80 illustrated comic.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Filter: A,B,D meet subject and page requirements; C fails subject. → Compute scores: A=2; B=1; D=3. → Choose best: Return D with both preference matches recorded. → **Deterministic output**: Reordering the **input** rows must not change D's unique winning score.

Point to each numbered **stage** and explain which recorded **input** it uses.

03. Filtering before scoring



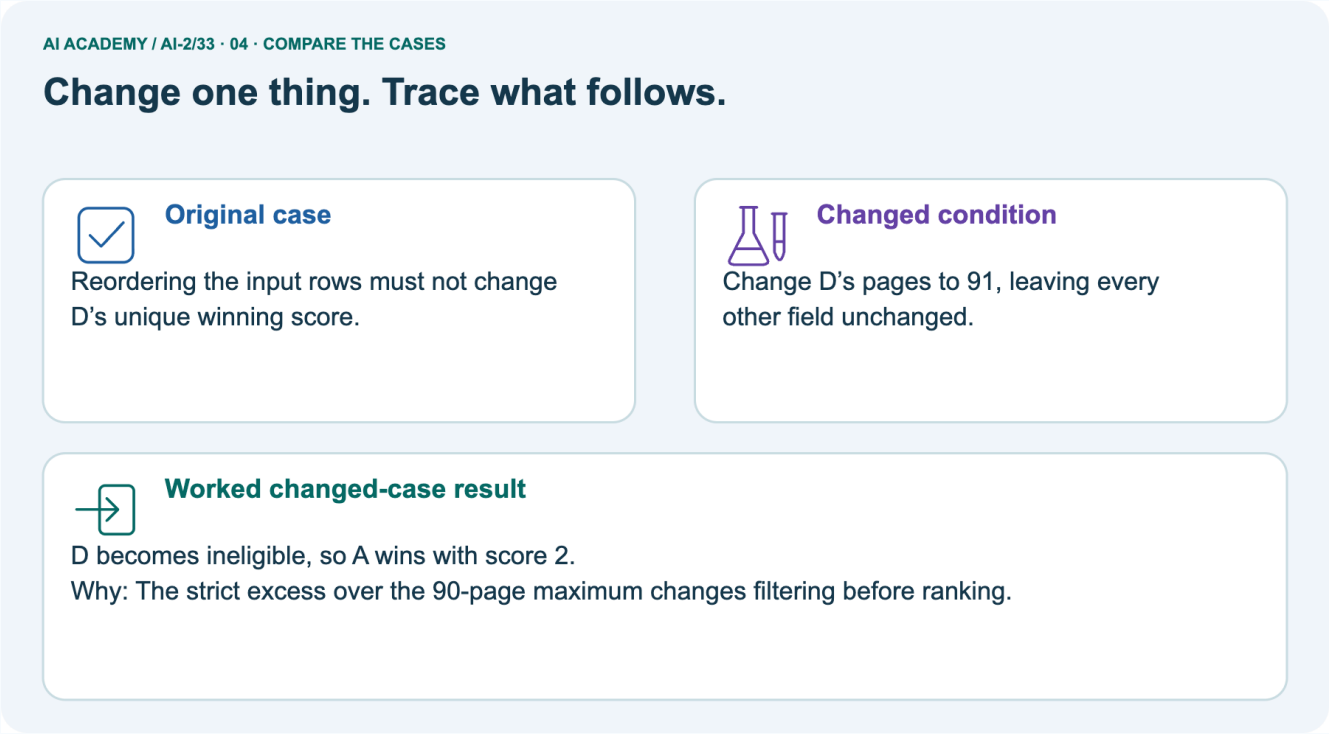
Follow the labelled picture with the matching explanation. Open the vector diagram

Filtering before scoring. A: Pages = 90, Preference score = 2, Status = eligible. B: Pages = 70, Preference score = 1, Status = eligible. C: Pages = 60, Preference score = 3, Status = subject fails.

D: Pages = 80, Preference score = 3, Status = eligible.

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



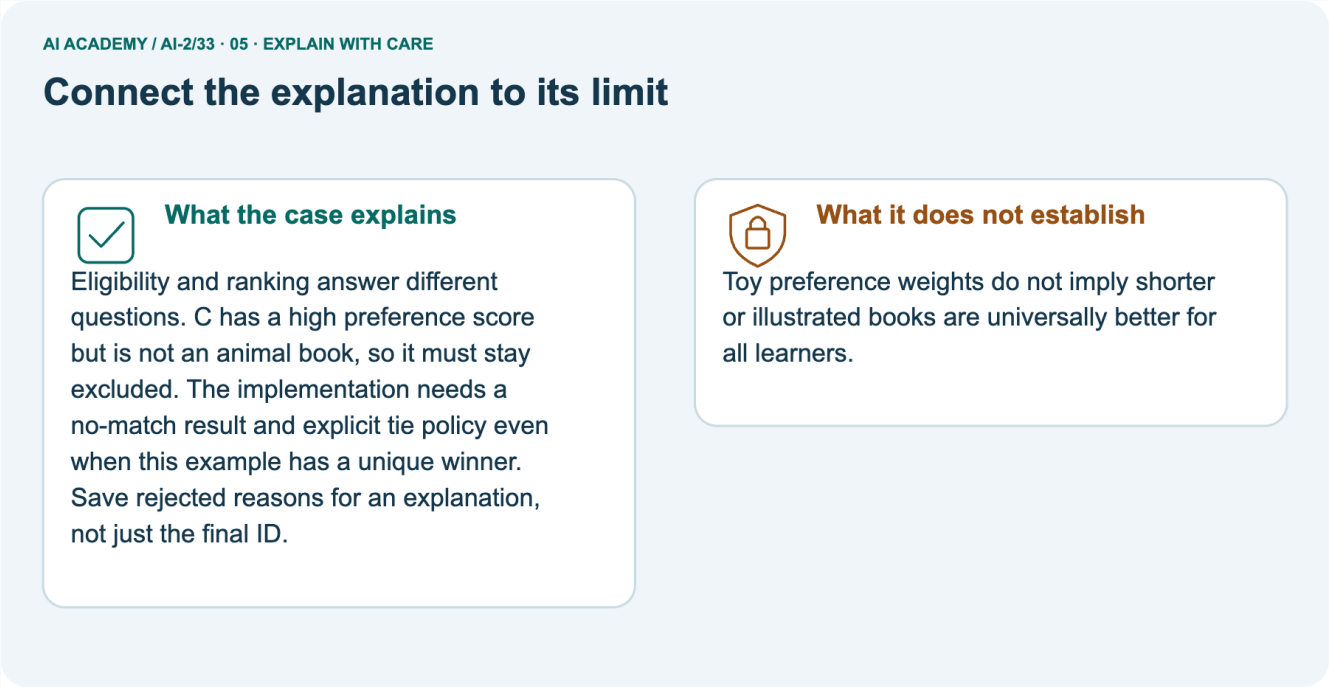
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Reordering the **input** rows must not change D's unique winning score. Changed condition: Change D's pages to 91, leaving every other field unchanged. Expected result: D becomes ineligible, so A wins with score 2.

Reason: The strict excess over the 90-page maximum changes filtering before ranking.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

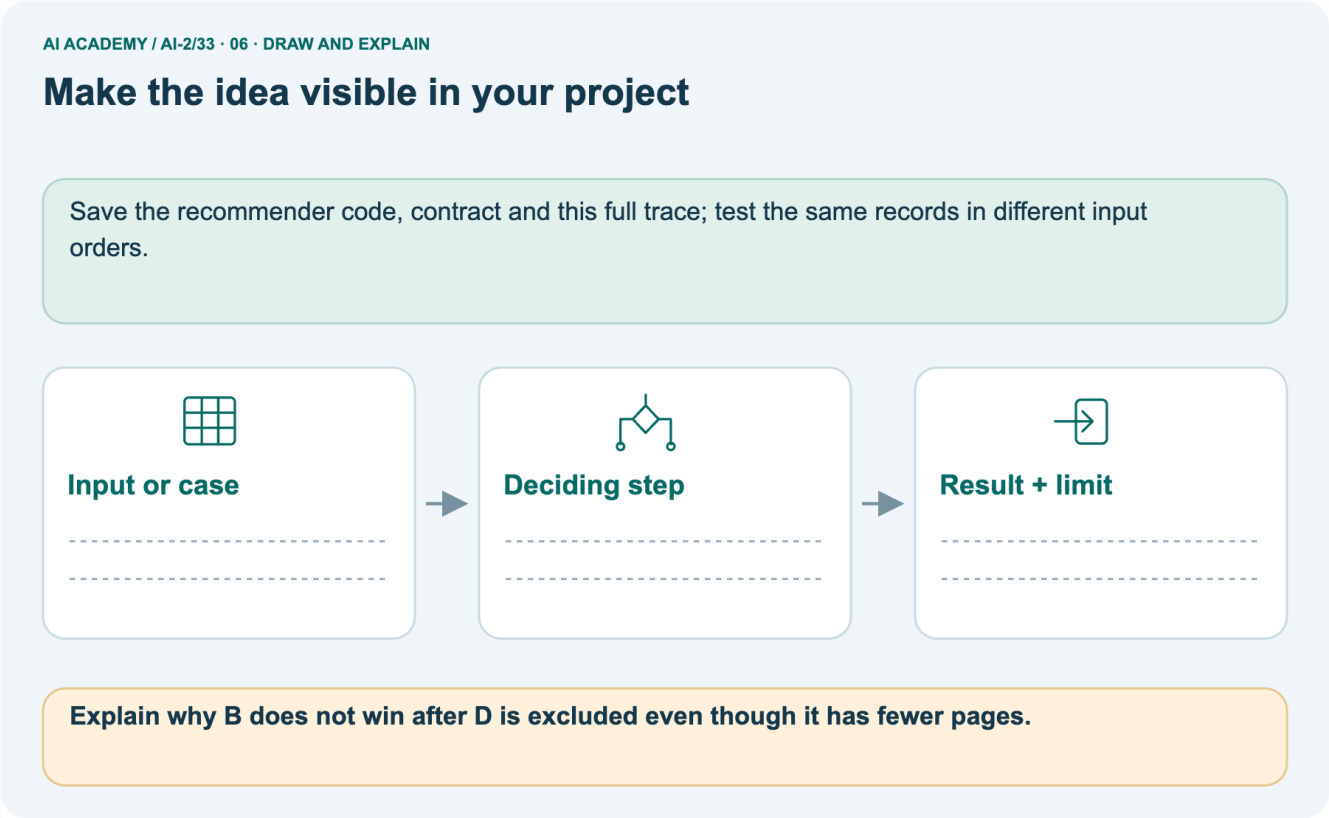
Eligibility and ranking answer different questions. C has a high preference score but is not an animal book, so it must stay excluded.

The implementation needs a no-match result and explicit tie policy even when this example has a unique winner. Save rejected reasons for an **explanation**, not just the final ID.

Limit: Toy preference weights do not imply shorter or illustrated books are universally better for all learners.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

Save the **recommender** code, contract and this full trace; test the same records in different **input** orders. Draw your **input**, deciding step, result and limit.

Supported question: Explain why B does not win after D is excluded even though it has fewer pages.

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Build the Recommender: the case

02. Follow the deciding steps

03. Filtering before scoring

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.